```java
@Target({ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Import({SchedulingConfiguration.class}) //注册ScheduledAnnotationBeanPostProcessor
public @interface EnableScheduling {}
```

```java
@Configuration(proxyBeanMethods = false)
@Role(2)
public class SchedulingConfiguration {
  public SchedulingConfiguration() {}
  @Bean( name = {"org.springframework.context.annotation.internalScheduledAnnotationProcessor"} )
  @Role(2)
  public ScheduledAnnotationBeanPostProcessor scheduledAnnotationProcessor() {
    return new ScheduledAnnotationBeanPostProcessor();
  }
}
```

```java
@EnableScheduling  //开启定时任务功能
@SpringBootApplication
public class SpringBootBasicApplication {
   public static void main(String[] args) {
       SpringApplication.run(SpringBootBasicApplication.class, args);
   }
}
```

SpringBoot启动

执行SpringBootBasicApplication.main() -> 创建SpringApplication实例，执行run()

- > 创建ApplicationContext（AnnotationConfigServletWebServerApplicationContext）
- > prepareContext（注册springBootBasicApplication的beanDefinition）
- > refreshContext
  - > invokeBeanFactoryPostProcessors（自动装配）
  - > onRefresh（创建WebServer）
  - > finishBeanFactoryInitialization
  - > finishRefresh（启动WebServer，提交定时任务）
- > callRunners

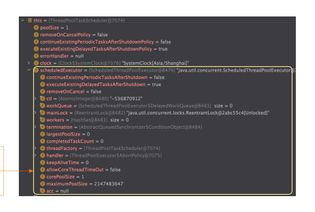
自动装配TaskSchedulingAutoConfiguration时，注册taskScheduler的beanDefinition

创建taskScheduler实例：

1. 加载ThreadPoolTaskScheduler.java
2. resolveBeforeInstantiation
3. doCreateBean
   - > createBeanInstance
   - > applyMergedBeanDefinitionPostProcessors
   - > populateBean 注入依赖
   - > initializeBean
     - > invokeInitMethods
   - > registerDisposableBeanIfNecessary

```java
void ThreadPoolTaskScheduler.super.afterPropertiesSet() → initialize() {
  this.executor = this.initializeExecutor(this.threadFactory, this.rejectedExecutionHandler);
}
```

```java
ExecutorService initializeExecutor(ThreadFactory threadFactory, RejectedExecutionHandler rejectedExecutionHandler) {
  //poolSize默认是1，可以通过配置文件更改
  this.scheduledExecutor = this.createExecutor(this.poolSize, threadFactory, rejectedExecutionHandler);
  return this.scheduledExecutor;
}
```



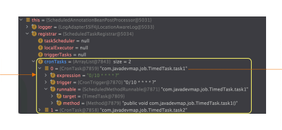
```
this = {ThreadPoolTaskScheduler@7074}
  poolSize = 1
  removeOnCancelPolicy = false
  continueExistingPeriodicTasksAfterShutdownPolicy = false
  executeExistingDelayedTasksAfterShutdownPolicy = true
  errorHandler = null
  clock = {Clock$SystemClock@7076} "SystemClock[Asia/Shanghai]"
  scheduledExecutor = {ScheduledThreadPoolExecutor@8476} "java.util.concurrent.ScheduledThreadPoolExecutor
    continueExistingPeriodicTasksAfterShutdown = false
    executeExistingDelayedTasksAfterShutdown = true
    removeOnCancel = false
    ctl = {AtomicInteger@8480} "-536870912"
    workQueue = {ScheduledThreadPoolExecutor$DelayedWorkQueue@8481}  size = 0
    mainLock = {ReentrantLock@8482} "java.util.concurrent.locks.ReentrantLock@2abc55c4[Unlocked]"
    workers = {HashSet@8483}  size = 0
    termination = {AbstractQueuedSynchronizer$ConditionObject@8484}
    largestPoolSize = 0
    completedTaskCount = 0
    threadFactory = {ThreadPoolTaskScheduler@7074}
    handler = {ThreadPoolExecutor$AbortPolicy@7075}
    keepAliveTime = 0
    allowCoreThreadTimeOut = false
    corePoolSize = 1
    maximumPoolSize = 2147483647
    acc = null
```

创建applicationTaskExecutor、timedTask实例：

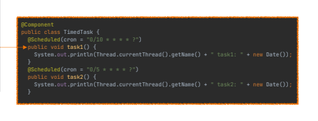
1. 加载com.javadevmap.job.TimedTask.java
2. resolveBeforeInstantiation
3. doCreateBean
   - > createBeanInstance
   - > applyMergedBeanDefinitionPostProcessors
   - > populateBean 注入依赖
   - > initializeBean（bean完成初始化之后，开始执行后置处理器）
   - > registerDisposableBeanIfNecessary

ScheduledAnnotationBeanPostProcessor.postProcessAfterInitialization()

> 解析TimedTask中带@Scheduled的方法，注册任务



```
this = {ScheduledAnnotationBeanPostProcessor@5031}
  logger = {LogAdapter$Slf4jLocationAwareLog@5033}
  registrar = {ScheduledTaskRegistrar@5034}
    taskScheduler = null
    localExecutor = null
    triggerTasks = null
    cronTasks = {ArrayList@7843}  size = 2
      0 = {CronTask@7859} "com.javadevmap.job.TimedTask.task1"
        expression = "0/10 * * * * ?"
        trigger = {CronTrigger@7870} "0/10 * * * * ?"
        runnable = {ScheduledMethodRunnable@7871} "com.javadevmap.job.TimedTask.task1"
          target = {TimedTask@7809}
          method = {Method@7879} "public void com.javadevmap.job.TimedTask.task1()"
      1 = {CronTask@7858} "com.javadevmap.job.TimedTask.task2"
```



```java
@Component
public class TimedTask {
  @Scheduled(cron = "0/10 * * * * ?")
  public void task1() {
    System.out.println(Thread.currentThread().getName() + " task1: " + new Date());
  }
  @Scheduled(cron = "0/5 * * * * ?")
  public void task2() {
    System.out.println(Thread.currentThread().getName() + " task2: " + new Date());
  }
```

```java
ScheduledAnnotationBeanPostProcessor.onApplicationEvent() → finishRegistration() {
  //search for TaskScheduler bean...
  registrar.setTaskScheduler((TaskScheduler)resolveSchedulerBean(beanFactory, TaskScheduler.class, false));
  //提交任务
  registrar.afterPropertiesSet();
}
```

```java
void ScheduledTaskRegistrar.scheduleTasks() {
  if (this.taskScheduler == null) {…}
  if (this.triggerTasks ≠ null) {}
  if (this.cronTasks ≠ null) { //遍历解析的jobs，提交给executor
    for (CronTask task : this.cronTasks) addScheduledTask(scheduleCronTask(task));
  }
  if (this.fixedRateTasks ≠ null) {}
  if (this.fixedDelayTasks ≠ null) {}
}
```

```java
ScheduledTask scheduleCronTask(CronTask task) {
  ScheduledTask scheduledTask = this.unresolvedTasks.remove(task); //获取scheduledTask
  scheduledTask.future = this.taskScheduler.schedule(task.getRunnable(), task.getTrigger());
  return null;
}
```

```java
ScheduledFuture<?> ThreadPoolTaskScheduler.schedule(Runnable task, Trigger trigger) {
  ScheduledExecutorService executor = this.getScheduledExecutor();
  //将task、executor等封装成可重复调度的Runnable对象
  return (new ReschedulingRunnable(task, trigger, this.clock, executor, errorHandler)).schedule();
}
```

```java
ScheduledFuture<?> ReschedulingRunnable.schedule() { //定时任务调度核心逻辑
  synchronized(this.triggerContextMonitor) {
    this.scheduledExecutionTime = this.trigger.nextExecutionTime(this.triggerContext);
    long initialDelay = this.scheduledExecutionTime.getTime() - this.triggerContext.getClock().millis();
    //提交到线程池执行
    this.currentFuture = this.executor.schedule(this, initialDelay, TimeUnit.MILLISECONDS);
    return this;
  }
}
```

```java
public ScheduledFuture<?> ScheduledThreadPoolExecutor.schedule(Runnable command, long delay, TimeUnit unit) {
  if (command == null || unit == null) throw new NullPointerException();
  RunnableScheduledFuture<?> t = decorateTask(command, new ScheduledFutureTask<Void>(command, null, triggerTime(delay, unit)));
  delayedExecute(t);
  return t;
}
```

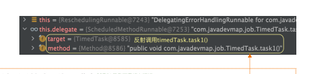
```java
//计算任务触发时间
private long triggerTime(long delay, TimeUnit unit) → triggerTime(unit.toNanos((delay < 0) ? 0 : delay)) {
  return now() + ((delay < (Long.MAX_VALUE >> 1)) ? delay : overflowFree(delay));
}
```

```java
ScheduledFutureTask(Runnable r, V result, long ns) {
  super(r, result) {
    this.callable = Executors.callable(runnable, result); //new RunnableAdapter<T>(task, result);
    this.state = NEW;
  }
  this.time = ns; //任务触发时间
  this.period = 0; //一次性任务
  this.sequenceNumber = sequencer.getAndIncrement();
}
```

```java
private void delayedExecute(RunnableScheduledFuture<?> task) {
  if (isShutdown()) reject(task); //线程池已关闭，执行决绝策略
  else {
    super.getQueue().add(task); //task入队：加入DelayedWorkQueue（小顶堆，保证queue[0]是最小的元素）
    if (isShutdown() && !canRunInCurrentRunState(task.isPeriodic()) && remove(task)) task.cancel(false);
    else ensurePrestart(); //确保至少1个worker在处理任务
  }
}
```

```java
void ensurePrestart(boolean mayInterruptIfRunning) {
  int wc = workerCountOf(ctl.get());
  if (wc < corePoolSize) addWorker(null, true); //scheduled thread pool的线程数永远不会超过corePoolSize
  else if (wc == 0) addWorker(null, false);
}
```

```java
//new Worker w > new Thread t > w加入线程池 > t.start() > 底层操作系统启动新线程后，回调t.run() > t.target.run() > w.run() > runWorker(this)
ThreadPoolExecutor.Worker.runWorker(Worker w) {
  Thread wt = Thread.currentThread(); Runnable task = w.firstTask;  w.firstTask = null;
  w.unlock(); //worker.state设置为0，允许中断
  boolean completedAbruptly = true;
  try {
    while (task ≠ null || (task = getTask()) ≠ null) { //自旋，不断地尝试从workQueue中取task
      w.lock(); //1. Worker加锁（state设置为1），防止正在执行的任务因其他线程调用shutdown()而中断
      if ((runStateAtLeast(ctl.get(), STOP) || (Thread.interrupted() && runStateAtLeast(ctl.get(), STOP))) && !wt.isInterrupted()) wt.interrupt();
      try {
        beforeExecute(wt, task);
        Throwable thrown = null;
        try {
          task.run(); //2. 执行用户定义的业务逻辑
        } catch (RuntimeException x) {…} } finally { afterExecute(task, thrown); }
      } finally {
        task = null;
        w.completedTasks++; //3. worker执行的task数加1
        w.unlock(); //4. woker解锁
      }
    }
    completedAbruptly = false;
  } finally {  processWorkerExit(w, completedAbruptly); //5. 关闭worker  }
}
```



```
this = {ReschedulingRunnable@7243} "DelegatingErrorHandlingRunnable for com.javad
this.delegate = {ScheduledMethodRunnable@7253} "com.javadevmap.job.TimedTask.ta
  target = {TimedTask@8585} 反射调用timedTask.task1()
  method = {Method@8586} "public void com.javadevmap.job.TimedTask.task1()"
```

```java
void ReschedulingRunnable.run() { //定时任务调度核心逻辑
  Date actualExecutionTime = new Date(this.triggerContext.getClock().millis());
  super.run(); //1.执行job
  Date completionTime = new Date(this.triggerContext.getClock().millis());
  synchronized(this.triggerContextMonitor) {
    this.triggerContext.update(this.scheduledExecutionTime, actualExecutionTime, completionTime);
    if (!this.obtainCurrentFuture().isCancelled()) {
      this.schedule(); //2.继续提交
    }
  }
}
```

```java
public void ScheduledFutureTask.run() {
  boolean periodic = isPeriodic(); //period ≠ 0
  if (!canRunInCurrentRunState(periodic)) cancel(false);
  //1. 一次性任务
  else if (!periodic) ScheduledFutureTask.super.run();
  //2. 定期任务：扔回queue里，继续执行
  else if (ScheduledFutureTask.super.runAndReset()) {
    setNextRunTime();
    //super.getQueue().add(task)
    reExecutePeriodic(outerTask);
  }
}
```